অধ্যায় ২১

২১

JavaScript Design Patterns, Software Architecture এবং Scalable Application Structure

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- একটি বড় হতে থাকা Code-এর সমস্যা চিনতে: জট পাকানো দায়িত্ব, লুকানো নির্ভরতা, পরীক্ষা করতে না পারা।
- Design Pattern ও Architecture-এর পার্থক্য বুঝতে, এবং কখন একটি Pattern দরকার নেই তা বলতে।
- Factory, Strategy, Observer, Repository, Adapter ও Decorator Pattern JavaScript-এর নিজস্ব ধরনে (Function ও Object দিয়ে) লিখতে।
- Dependency Injection ও Composition Root দিয়ে অংশগুলো আলাদা রাখতে।
- Layered Architecture-এ নির্ভরতার দিক ঠিক করতে, এবং একটি Test দিয়ে সেই নিয়ম পাহারা দিতে।
- একটি Workflow-কে টেবিলভিত্তিক State Machine হিসেবে প্রকাশ করতে।
- Contract Test দিয়ে একই চুক্তির একাধিক Implementation যাচাই করতে।
- একটি সম্পূর্ণ **VisaFlow** Application তৈরি করতে, যা Node.js ও Browser—দুই পরিবেশে একই Domain ও Service ব্যবহার করে।

পূর্বশর্ত

- অধ্যায় ১২: ES Module, `import` / `export` ।
- অধ্যায় ১৩: Class, Composition বনাম Inheritance।
- অধ্যায় ১৬: Error-এর `code` , Repository ও Service, Test Double।
- অধ্যায় ১৮: Immutable হালনাগাদ।
- অধ্যায় ২০: Allowlist, উত্তরাধিকারের Key-এর ঝুঁকি।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ৫০ লাইনের Script-এ সবকিছু এক জায়গায় লিখলে সমস্যা নেই। কিন্তু ছয় মাস পর সেটি ৫,০০০ লাইন, তিনজন Developer, এবং প্রতিটি নতুন Feature-এ পুরোনো কিছু ভেঙে যায়। কারণ প্রায় সব সময় একই: একটি Function একসঙ্গে Form পড়ে, দাম হিসাব করে, `localStorage`-এ লেখে এবং DOM আঁকে। তখন দামের নিয়ম বদলাতে DOM-এর Code ছুঁতে হয়, এবং দামের হিসাব Browser ছাড়া Test করা যায় না।

Architecture-এর লক্ষ্য তাই সুন্দর Diagram নয়; লক্ষ্য হলো **পরিবর্তন সস্তা রাখা**। যে অংশগুলো ভিন্ন কারণে বদলায় (ব্যবসার নিয়ম, সংরক্ষণের পদ্ধতি, পর্দার নকশা), সেগুলো আলাদা থাকলে একটি বদলালে অন্যটি অক্ষত থাকে। Design Pattern সেই আলাদা করার পরীক্ষিত, নামযুক্ত উপায়—এবং নাম থাকায় দল সহজে আলোচনা করতে পারে: "এখানে একটি Strategy লাগবে।"

২১.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
ES Module (অধ্যায় ১২)	Module-কে স্তরে সাজানো, নির্ভরতার দিক, Module-Singleton-এর ঝুঁকি
Composition বনাম Inheritance (অধ্যায় ১৩)	Strategy ও Decorator: Composition-এর নামযুক্ত রূপ
Repository ও Service (অধ্যায় ১৬, BookingGuard)	Adapter, Contract Test, Composition Root
Event Listener (অধ্যায় ৯)	নিজস্ব Event Bus, Observer, Error-এর বিচ্ছিন্নতা
`if/switch`-এ Status (আগের Project)	টেবিলভিত্তিক State Machine
—	Architecture Test, Dependency Injection-এর পূর্ণ প্রয়োগ

অধ্যায় ১৩ ও ১৬-এর Project দুটি ছিল Booking-কেন্দ্রিক। পুনরাবৃত্তি এড়াতে এই অধ্যায়ের Project ভিন্ন একটি Domain-এ: Visa আবেদনের Workflow।

২১.২ Design Pattern বনাম Architecture

	Design Pattern	Architecture
পরিসর	কয়েকটি Function বা Object	পুরো Application
প্রশ্ন	এই নির্দিষ্ট সমস্যার পরিচিত সমাধান কী?	অংশগুলো কীভাবে ভাগ, কে কার ওপর নির্ভর করে?
উদাহরণ	Strategy, Observer, Factory	Layered, Hexagonal, Event-driven
ভুল সিদ্ধান্তের মূল্য	স্থানীয়, সহজে বদলানো যায়	ব্যাপক, বদলানো ব্যয়বহুল

Pattern-এর বেশিরভাগ বর্ণনা Java-র মতো Class-নির্ভর ভাষার জন্য লেখা। JavaScript-এ First-class Function, Object Literal ও Closure থাকায় অনেক Pattern কয়েক লাইনে হয়ে যায়; এ অধ্যায়ে আমরা সেই হালকা রূপই ব্যবহার করব।

> **সতর্কতা:** Pattern একটি সমস্যার সমাধান, লক্ষ্য নয়। একটি মাত্র Fee-র নিয়মের জন্য Strategy, অথবা একটি মাত্র Listener-এর জন্য Event Bus অপ্রয়োজনীয় জটিলতা। নিয়ম: **একই ধরনের দ্বিতীয় বা তৃতীয় প্রয়োজন এলে** Pattern-এ রূপান্তর করুন (প্রায়ই "Rule of Three" বলা হয়)।

২১.৩ Factory: তৈরির নিয়ম এক জায়গায়

Factory একটি Function, যা Object তৈরি করে এবং তৈরির সব নিয়ম—যাচাই, Default মান, কোন ধরনের Object—নিজের ভেতরে রাখে। ব্যবহারকারী Code জানে না, জানতেও চায় না, ভেতরে কোন Implementation।

factory.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createNotifier(channel) {
  const notifiers = {
    sms: { send: (to, text) => `SMS ${to}: ${text.slice(0, 40)}` },
    email: { send: (to, text) => `Email ${to}: ${text}` },
    console: { send: (to, text) => `LOG ${to}: ${text}` },
  };
  if (!Object.hasOwn(notifiers, channel)) {
    throw new Error(`অজানা Channel: ${channel}`);
  }
  return notifiers[channel];
}

for (const channel of ['sms', 'email']) {
  const notifier = createNotifier(channel);
  console.log(notifier.send('rafi', 'আপনার Visa আবেদন জমা হয়েছে'));
}
try {
  createNotifier('fax');
} catch (error) {
  console.log(error.message);
}
```

Output

```
SMS rafi: আপনার Visa আবেদন জমা হয়েছে
Email rafi: আপনার Visa আবেদন জমা হয়েছে
অজানা Channel: fax
```

ব্যবহারকারী Code প্রতিটি Notifier-কে একইভাবে ডাকে (`send`)—এটিই Polymorphism, Class ছাড়াই। নতুন Channel যোগ করতে শুধু Factory বদলায়। Project-এর `createApplication` আরেক ধরনের Factory: এটি নিশ্চিত করে যে একটি আবেদন **কখনো অবৈধ অবস্থায় তৈরি হতে পারে না**—সব যাচাই পেরোলে তবেই Object।

২১.৪ Strategy: if/else-এর শিকলের বদলে বিনিময়যোগ্য নিয়ম

যখন একই কাজের কয়েকটি বিকল্প নিয়ম থাকে এবং নতুন নিয়ম যোগ হতেই থাকে, তখন প্রতিটি নিয়মকে একই আকারের একটি Object বা Function-এ রাখুন, এবং চলার সময় বেছে নিন। একে বলে **Strategy**।

strategy.js · JavaScript — চলবে: Browser Console ও Node.js

```
const discountStrategies = {
  none: () => 0,
  earlyBird: ({ daysAhead, price }) => (daysAhead >= 60 ? price * 0.1 : 0),
  group: ({ travelers, price }) => (travelers >= 5 ? price * 0.15 : 0),
};

function finalPrice(order, strategyName) {
  const strategy = discountStrategies[strategyName] ?? discountStrategies.none;
  return order.price - Math.round(strategy(order));
}

const order = { price: 40000, travelers: 5, daysAhead: 75 };
for (const name of ['none', 'earlyBird', 'group', 'unknown']) {
  console.log(name.padEnd(10), finalPrice(order, name));
}

// নতুন নিয়ম: পুরোনো কোনো লাইন না বদলে যোগ।
discountStrategies.student = ({ price }) => price * 0.2;
console.log('student'.padEnd(10), finalPrice(order, 'student'));
```

Output

```
none       40000
earlyBird  36000
group      34000
unknown    40000
student    32000
```

`finalPrice` কোনো নিয়মের বিস্তারিত জানে না; তার কোনো `if (type === 'group')` নেই। এটি **Open/Closed Principle**-এর ব্যবহারিক রূপ: নতুন আচরণের জন্য খোলা, পুরোনো Code বদলানোর জন্য বন্ধ। একটি সতর্কতা: এখানে `unknown`-এর জন্য Default নিয়ম নেওয়া হয়েছে; ব্যবসার নিয়মে অজানা নাম অনেক সময় Error হওয়াই উচিত (Project-এর `policyFor` তাই করে), এবং Key খোঁজায় `??`-এর আগে `Object.hasOwn` দিয়ে উত্তরাধিকারের নাম আটকানো নিরাপদ (অধ্যায় ২০)।

২১.৫ Observer: Publisher জানে না কে শুনছে

Observer (বা Publish-Subscribe) Pattern-এ একটি অংশ ঘটনা ঘোষণা করে, এবং আগ্রহী অংশগুলো নিজেরাই নিবন্ধন করে। ঘোষণাকারী জানে না কতজন শুনছে, বা তারা কী করে। Browser-এর `addEventListener` এই Pattern-ই।

observer.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createEventBus() {
  const handlers = new Map();
  return {
    on(type, handler) {
      if (!handlers.has(type)) {
        handlers.set(type, new Set());
      }
      handlers.get(type).add(handler);
      return () => handlers.get(type).delete(handler);
    },
    emit(type, payload) {
      for (const handler of [...(handlers.get(type) ?? [])]) {
        handler(payload);
      }
    },
  };
}

const bus = createEventBus();
const stopEmail = bus.on('approved', (id) => {
  console.log(`Email: ${id} অনুমোদিত`);
});
bus.on('approved', (id) => console.log(`Report হালনাগাদ: ${id}`));

bus.emit('approved', 'VF-001');
stopEmail();
bus.emit('approved', 'VF-002');
```

Output

```
Email: VF-001 অনুমোদিত
Report হালনাগাদ: VF-001
Report হালনাগাদ: VF-002
```

`on` একটি Unsubscribe Function ফেরত দেয়—অধ্যায় ১৯-এর Memory Leak এড়ানোর অভ্যাস। Observer-এর মূল্য: প্রবাহ লুকিয়ে যায়। Code পড়ে বোঝা কঠিন যে `emit`-এর পর কী কী ঘটবে, এবং একটি Listener-এর Error পুরো `emit` থামিয়ে দিতে পারে। Project-এর Event Bus তাই প্রতিটি Handler আলাদা `try/catch`-এ চালায়। নিয়ম:

Event ব্যবহার করুন "ঘটে গেছে" জানাতে (`application:changed`), "এটা করো" আদেশের জন্য নয়—আদেশের জন্য সরাসরি Function Call অনেক স্পষ্ট।

২১.৬ Repository, Adapter এবং Dependency Injection

Repository তথ্য কোথায় ও কীভাবে থাকে তা লুকায়: Service শুধু জানে `save` , `get` , `list` । পেছনে Memory, `localStorage` , না Server-এর API—তা Repository-র বিষয়। একটি বাইরের Interface-কে (যেমন `localStorage` -এর `getItem` / `setItem`) আমাদের চাওয়া Interface-এ রূপান্তর করাকে বলে **Adapter**।

Dependency Injection (DI) মানে একটি অংশ তার নির্ভরতা নিজে তৈরি বা Import করে না; বাইরে থেকে পায়। এতে Service-কে একটি নকল Repository দিয়ে Test করা যায়, এবং Browser ও Node.js-এ ভিন্ন Repository দিয়ে চালানো যায়।

di.js · JavaScript — চলবে: Browser Console ও Node.js

```
function createGreetingService({ repository, clock }) {
  return {
    greet(id) {
      const user = repository.find(id);
      const hour = clock().getHours();
      const part = hour < 12 ? 'সুপ্রভাত' : 'শুভ সন্ধ্যা';
      return user ? `${part}, ${user.name}` : 'অজানা ব্যবহারকারী';
    },
  };
}

// Test-এর জন্য: স্থির তথ্য ও স্থির ঘড়ি।
const fakeRepository = { find: (id) => (id === 1 ? { name: 'Mina' } : null) };
const morning = () => new Date(2026, 9, 15, 9, 0);
const evening = () => new Date(2026, 9, 15, 19, 0);

const at = (clock) =>
  createGreetingService({ repository: fakeRepository, clock });
console.log(at(morning).greet(1));
console.log(at(evening).greet(1));
console.log(at(morning).greet(2));
```

Output

```
সুপ্রভাত, Mina
শুভ সন্ধ্যা, Mina
অজানা ব্যবহারকারী
```

এখানে কোনো DI Framework নেই—শুধু একটি Object Parameter। ঘড়িও একটি নির্ভরতা: `new Date()` ভেতরে লিখলে সকাল-সন্ধ্যার আচরণ Test করতে দিনের নির্দিষ্ট সময়ের জন্য অপেক্ষা করতে হতো। এখন একই Test যেকোনো সময়ে একই ফল দেয়।

সব নির্ভরতা কোথাও না কোথাও সত্যিই তৈরি ও জোড়া লাগাতে হয়। সেই একমাত্র জায়গার নাম **Composition Root**—সাধারণত Application-এর প্রবেশ File (Project-এ `main.mjs` ও `demo.mjs`)। বাকি সব Code শুধু Parameter পায়।

২১.৭ Module-Singleton: সুবিধা ও ঝুঁকি

ES Module একবারই চলে, এবং প্রতিটি `import` একই Module-এর একই অবস্থা পায়। তাই Module-এর স্তরে একটি Variable কার্যত একটি **Singleton**:

counter.mjs · JavaScript — চলবে: Node.js

```
let count = 0;
export const increment = () => {
  count += 1;
  return count;
};
```

main.mjs · JavaScript — চলবে: Node.js

```
import { increment } from './counter.mjs';

const again = await import('./counter.mjs');
increment();
console.log(again.increment(), again.increment === increment);
```

Output

```
2 true
```

দুটি ভিন্ন `import` একই `count` ভাগ করে। একটি Logger বা Configuration-এর জন্য এটি সুবিধাজনক। কিন্তু ঝুঁকিও এখানে: এক Test-এ বদলানো অবস্থা পরের Test-এ রয়ে যায়, এবং যে Code এমন Module Import করে, তাকে অন্য Implementation দিয়ে চালানো যায় না। তাই অবস্থাযুক্ত জিনিস (Repository, Event Bus) Project-এ Factory Function দিয়ে তৈরি এবং Inject করা হয়েছে, Module-এর স্তরে নয়।

২১.৮ State Machine: Workflow-কে তথ্য হিসেবে

একটি Visa আবেদন যেকোনো অবস্থা থেকে যেকোনো অবস্থায় যেতে পারে না: "অনুমোদিত" আবেদন আবার "জমা" হয় না। এই নিয়ম `if/else`-এ ছড়িয়ে লিখলে দ্রুত অসামঞ্জস্য তৈরি হয়। পরিবর্তে পুরো Workflow-কে একটি টেবিলে রাখুন: প্রতিটি অবস্থায় কোন কাজ চলে, এবং কাজের পর নতুন অবস্থা।

অবস্থা	কাজ	নতুন অবস্থা
draft	submit / withdraw	submitted / withdrawn
submitted	requestDocuments / approve / reject / withdraw	documents-requested / approved / rejected / withdrawn
documents-requested	resubmit / withdraw	submitted / withdrawn
approved, rejected, withdrawn	কোনোটি নয়	চূড়ান্ত

টেবিলটিই Code (অনুচ্ছেদ ২১.১০-এর `workflow.mjs`)। এর তিনটি সুবিধা: অবৈধ পরিবর্তন এক জায়গায় আটকায়; UI টেবিল থেকেই জানে কোন Button দেখাতে হবে (`availableActions`), তাই UI ও নিয়ম কখনো অমিল হয় না; এবং ব্যবসার মানুষ টেবিলটি পড়ে যাচাই করতে পারেন। এর চেয়ে জটিল প্রয়োজনে (শর্তযুক্ত পরিবর্তন, সময়সীমা, Nested অবস্থা) XState-এর মতো Library আছে।

২১.৯ Layered Architecture এবং নির্ভরতার দিক

Project-এর Code চারটি স্তরে:

স্তর	Folder	দায়িত্ব	নির্ভর করতে পারে
Domain	`src/domain`	ব্যবসার নিয়ম: আবেদন, Fee, Workflow	কিছুর ওপর নয়
Application	`src/app`	Use Case: তৈরি, কাজ, সারাংশ; Event	Domain
Infrastructure	`src/infra`	সংরক্ষণ: Memory, Web Storage	কোনো স্তর নয় (শুধু চুক্তি মানে)
UI	`src/ui`	পর্দায় আঁকা	Domain
Composition Root	`src/main.mjs`, `demo.mjs`	সব জোড়া লাগানো	সব

মূল নিয়ম: **নির্ভরতা ভেতরের দিকে, ব্যবসার নিয়মের দিকে।** Domain জানে না Browser আছে কি না, `localStorage` আছে কি না; তাই একই Domain Node.js-এর Test-এ, Browser-এ এবং ভবিষ্যতে একটি Server-এ হুবহু চলে। Service Repository-র ওপর নির্ভর করে, কিন্তু কোনো নির্দিষ্ট Repository Import করে না—সে শুধু চুক্তি (`save`, `get`, `list`)

জানে, এবং Composition Root একটি বাস্তব Repository পাঠায়। একে বলে **Dependency Inversion**। এই ধারণার বিস্তৃত রূপগুলোর নাম Hexagonal (Ports and Adapters) বা Clean Architecture।

নিয়ম শুধু লিখে রাখলে সময়ের সঙ্গে ভাঙে: কেউ তাড়াহুড়োয় Domain-এ একটি `localStorage` লিখে ফেলেন। Project-এর **Architecture Test** প্রতিটি File-এর `import` পড়ে নিয়মটি যাচাই করে, তাই নিয়ম ভাঙলে Test ব্যর্থ হয়।

২১.১০ Project: VisaFlow

VisaFlow একটি ভ্রমণ সংস্থার Visa আবেদনের Workflow Tracker। কর্মী আবেদন তৈরি করেন (আবেদনকারী, দেশ, Visa-র ধরন, যাত্রী, Express), এবং আবেদনটি জমা, কাগজ চাওয়া, অনুমোদন, প্রত্যাখ্যান বা প্রত্যাহারের পথে এগোয়। Fee আসে Visa-র ধরনের Strategy থেকে। একই Domain ও Service দুই Composition Root-এ চলে: Browser-এ (`localStorage`-এ সংরক্ষণ, DOM UI) এবং Node.js-এ (Memory-তে, Console-এ)।

> **নোট:** VisaFlow একটি শিক্ষামূলক Demo। Fee-গুলো কাল্পনিক; এতে Authentication, Authorization, Server বা ভাগ করা Database নেই, এবং `localStorage`-এর তথ্য ব্যবহারকারী নিজেই বদলাতে পারেন (অধ্যায় ২০)। বাস্তব System-এ Domain ও Service একই রেখে একটি Server-ভিত্তিক Repository Adapter ও Authorization যোগ করতে হবে—এই Architecture-এর সুবিধা ঠিক সেখানেই।

File-এর কাঠামো

Folder-এর কাঠামো

```
visa-flow/
├── index.html, style.css
├── demo.mjs                  Node.js Composition Root
├── src/
│   ├── main.mjs              Browser Composition Root
│   ├── domain/
│   │   ├── errors.mjs        DomainError
│   │   ├── fees.mjs          Fee-র Strategy ও Express Decorator
│   │   ├── workflow.mjs      State Machine
│   │   └── application.mjs   আবেদনের Factory
│   ├── app/
│   │   ├── events.mjs        Event Bus (Observer)
│   │   ├── ids.mjs           ক্রমিক ID
│   │   └── service.mjs       Use Case (DI)
│   ├── infra/
│   │   ├── memory-repository.mjs
│   │   └── storage-repository.mjs   Web Storage Adapter
│   └── ui/
│       └── view.mjs          DOM-এ আঁকা
└── tests/                    domain, service, repositories, architecture
```

Domain: errors.mjs ও fees.mjs

errors.mjs · JavaScript চলবে: Node.js

```
// Domain-এর নিয়ম ভাঙলে এই Error; code দিয়ে UI বা Test সিদ্ধান্ত নেয়।
export class DomainError extends Error {
  constructor(code, message) {
    super(message);
    this.name = 'DomainError';
    this.code = code;
  }
}
```

fees.mjs · JavaScript চলবে: Node.js

```
import { DomainError } from './errors.mjs';

// Strategy: প্রতিটি Visa-র ধরনের নিজস্ব Fee-র নিয়ম, একই আকারের Object-এ।
const tourist = {
  label: 'Tourist',
  maxTravelers: 6,
  fee: ({ travelers }) => 3000 * travelers,
};

const business = {
  label: 'Business',
  maxTravelers: 3,
  fee: ({ travelers }) => 5500 * travelers + 1000,
};

const student = {
  label: 'Student',
  maxTravelers: 1,
  fee: () => 2500,
};

export const FEE_POLICIES = Object.freeze({ tourist, business, student });

// একটি Strategy-কে মুড়ে আরেকটি: মূল নিয়ম না বদলে Express-এর ৪০% যোগ।
export const withExpress = (policy) => ({
  ...policy,
  fee: (input) => Math.round(policy.fee(input) * 1.4),
});

export function policyFor(visaType, express = false) {
  if (!Object.hasOwn(FEE_POLICIES, visaType)) {
    throw new DomainError('UNKNOWN_TYPE', `অজানা Visa-র ধরন: ${visaType}`);
  }
  const policy = FEE_POLICIES[visaType];
```

```
  return express ? withExpress(policy) : policy;
}
```

তিনটি Strategy একই আকারের: `label`, `maxTravelers`, `fee`। `withExpress` একটি **Decorator**: মূল Strategy-কে মুড়ে নতুন একটি দেয়, যার আচরণ একটু বাড়ানো, কিন্তু মূলটি অপরিবর্তিত। Express আলাদা Strategy না হয়ে Decorator হওয়ায় তিন ধরন × Express-এর জন্য ছয়টি Object লিখতে হয়নি। `Object.hasOwn` উত্তরাধিকারের নাম (`toString`) Visa-র ধরন হিসেবে আটকায়।

Domain: workflow.mjs

workflow.mjs · JavaScript চলবে: Node.js

```
import { DomainError } from './errors.mjs';

// State Machine: কোন অবস্থায় কোন কাজ চলে, এবং কাজের পর নতুন অবস্থা।
export const WORKFLOW = Object.freeze({
  draft: Object.freeze({ submit: 'submitted', withdraw: 'withdrawn' }),
  submitted: Object.freeze({
    requestDocuments: 'documents-requested',
    approve: 'approved',
    reject: 'rejected',
    withdraw: 'withdrawn',
  }),
  'documents-requested': Object.freeze({
    resubmit: 'submitted',
    withdraw: 'withdrawn',
  }),
  approved: Object.freeze({}),
  rejected: Object.freeze({}),
  withdrawn: Object.freeze({}),
});

// শুধু নিজের Key: "toString"-এর মতো উত্তরাধিকারের নাম কখনো কাজ নয়।
const movesFrom = (status) =>
  (Object.hasOwn(WORKFLOW, status) ? WORKFLOW[status] : {});

export const availableActions = (status) => Object.keys(movesFrom(status));

export const isFinal = (status) => availableActions(status).length === 0;

// নতুন Object ফেরত দেয়; পুরোনোটি অপরিবর্তিত (অধ্যায় ১৮)।
export function applyAction(application, action, now) {
  const moves = movesFrom(application.status);
  if (!Object.hasOwn(moves, action)) {
    throw new DomainError('INVALID_TRANSITION',
      `${application.status} অবস্থায় "${action}" করা যায় না`);
  }
  const status = moves[action];
```

```
  const entry = Object.freeze({ status, action, at: now.toISOString() });
  return Object.freeze({
    ...application,
    status,
    history: Object.freeze([...application.history, entry]),
  });
}
```

অনুচ্ছেদ ২১.৮-এর টেবিল এখানে হুবহু Object। `applyAction` নতুন Frozen Object ফেরত দেয় এবং ইতিহাসে একটি Entry যোগ করে—পুরোনো আবেদন অপরিবর্তিত, তাই কে কখন কী করেছে তা পুরোটা থেকে যায়। সময় (`now`) Parameter-এ আসে; Domain নিজে ঘড়ি পড়ে না।

Domain: application.mjs

application.mjs · JavaScript — চলবে: Node.js

```
import { DomainError } from './errors.mjs';
import { policyFor } from './fees.mjs';

export const COUNTRIES = Object.freeze([
  'Thailand', 'Malaysia', 'Japan', 'Turkey',
]);

// Factory: যাচাই, স্বাভাবিকীকরণ ও হিসাব এক জায়গায়; ফল সব সময় বৈধ ও Frozen।
// ID নেওয়া হয় শুধু সব যাচাই পেরোলে, যাতে ব্যর্থ চেষ্টায় ID নষ্ট না হয়।
export function createApplication(input, { newId, now }) {
  const applicant = String(input.applicant ?? '').trim().replace(/\s+/g, ' ');
  if (applicant.length < 2 || applicant.length > 60) {
    throw new DomainError('INVALID_APPLICANT', 'আবেদনকারীর নাম ২-৬০ অক্ষর');
  }
  if (!COUNTRIES.includes(input.country)) {
    throw new DomainError('INVALID_COUNTRY', `অসমর্থিত দেশ: ${input.country}`);
  }
  const express = input.express === true;
  const policy = policyFor(input.visaType, express);
  const { travelers } = input;
  if (!Number.isInteger(travelers) || travelers < 1
    || travelers > policy.maxTravelers) {
    throw new DomainError('INVALID_TRAVELERS',
      `${policy.label} Visa-য় সর্বোচ্চ ${policy.maxTravelers} জন যাত্রী`);
  }
  return Object.freeze({
    id: newId(),
    applicant,
    country: input.country,
    visaType: input.visaType,
    travelers,
    express,
```

```
    fee: policy.fee({ travelers }),
    status: 'draft',
    history: Object.freeze([
      Object.freeze({
        status: 'draft', action: 'create', at: now.toISOString(),
      }),
    ]),
  });
}
```

Factory নাম স্বাভাবিক করে (বাড়তি ফাঁকা সরায়), দেশ Allowlist-এ মেলায়, Strategy থেকে যাত্রীর সীমা নেয় (Student Visa-য় একজন), এবং Fee হিসাব করে। `express === true`: শুধু আসল `true`; `"yes"` বা `1` নয়। ID-র উৎস (`newId`) ও সময় বাইরে থেকে আসে, তাই Factory পরিবেশ-মুক্ত; এবং `newId()` ডাকা হয় শুধু সব যাচাই পেরোনোর পর, যাতে একটি ব্যর্থ চেষ্টা একটি ID নষ্ট না করে।

Application: events.mjs, ids.mjs ও service.mjs

events.mjs · JavaScript — চলবে: Node.js

```
// Observer: Publisher জানে না কে শুনছে; Subscriber-রা একে অপরকে চেনে না।
export function createEventBus({ onError = () => {} } = {}) {
  const handlers = new Map();

  function on(type, handler) {
    if (!handlers.has(type)) {
      handlers.set(type, new Set());
    }
    handlers.get(type).add(handler);
    return () => handlers.get(type).delete(handler);
  }

  function emit(type, payload) {
    // Copy-র ওপর Loop: কোনো Handler নিজেকে সরালেও ক্রম ঠিক থাকে।
    for (const handler of [...(handlers.get(type) ?? [])]) {
      try {
        handler(payload);
      } catch (error) {
        onError(error, type);
      }
    }
  }

  const count = (type) => handlers.get(type)?.size ?? 0;
  return { on, emit, count };
}
```

ids.mjs · JavaScript চলবে: Node.js

```
// ক্রমিক ID: VF-001, VF-002, ... (Test-এ নিশ্চিত ফলের জন্য Inject করা হয়)।
export function createSequence(prefix, start = 1) {
  let next = start;
  return () => `${prefix}-${String(next++).padStart(3, '0')}`;
}

// সংরক্ষিত ID-গুলোর সবচেয়ে বড় সংখ্যার পর থেকে। সংখ্যা গুনে শুরু করলে
// মাঝে ফাঁক থাকলে একটি পুরোনো ID আবার আসত, এবং আগের আবেদন মুছে যেত।
export function sequenceAfter(prefix, ids) {
  const numbers = ids.map((id) => Number(id.slice(prefix.length + 1)))
    .filter(Number.isInteger);
  return createSequence(prefix, Math.max(0, ...numbers) + 1);
}
```

service.mjs · JavaScript চলবে: Node.js

```
import { createApplication } from '../domain/application.mjs';
import { applyAction } from '../domain/workflow.mjs';
import { DomainError } from '../domain/errors.mjs';

// Application Service: Use Case-গুলো সমন্বয় করে। কোন Repository বা কোন UI,
// তা জানে না; সব নির্ভরতা Constructor-এ আসে (Dependency Injection)।
export function createVisaService(deps) {
  const { repository, events, nextId, clock = () => new Date() } = deps;

  function create(input) {
    const context = { newId: nextId, now: clock() };
    const application = createApplication(input, context);
    repository.save(application);
    const { id, status, fee } = application;
    events.emit('application:created', { id, status, fee });
    return application;
  }

  function act(id, action) {
    const current = repository.get(id);
    if (!current) {
      throw new DomainError('NOT_FOUND', `${id} নামে কোনো আবেদন নেই`);
    }
    const next = applyAction(current, action, clock());
    repository.save(next);
    events.emit('application:changed',
      { id, action, from: current.status, to: next.status });
    return next;
  }

  function summary() {
    const counts = {};
```

```
    let approvedFees = 0;
    for (const application of repository.list()) {
      counts[application.status] = (counts[application.status] ?? 0) + 1;
      if (application.status === 'approved') {
        approvedFees += application.fee;
      }
    }
    return { counts, approvedFees };
  }

  return { create, act, summary, list: () => repository.list() };
}
```

Service-এর প্রতিটি Use Case একই ছন্দে: পড়া, Domain-কে সিদ্ধান্ত নিতে দেওয়া, সংরক্ষণ, ঘোষণা। ব্যবসার কোনো নিয়ম এখানে নেই—সব Domain-এ। লক্ষ করুন, `act`-এ Domain Error ছুড়লে `save` ও `emit` কোনোটিই চলে না: ব্যর্থ কাজ কোনো চিহ্ন রেখে যায় না। Service জানে না কোন Repository বা কতজন Subscriber।

Infrastructure: দুটি Repository

memory-repository.mjs · JavaScript — চলবে: Node.js

```
// Repository-র চুক্তি: save(application), get(id) বা null, list() id-র ক্রমে।
// প্রতিটি Method Copy দেয়-নেয়, যাতে বাইরের পরিবর্তন ভেতরে না পৌঁছায়।
export function createMemoryRepository() {
  const items = new Map();
  return {
    save(application) {
      items.set(application.id, structuredClone(application));
    },
    get(id) {
      return items.has(id) ? structuredClone(items.get(id)) : null;
    },
    list() {
      return [...items.values()]
        .map((item) => structuredClone(item))
        .toSorted((a, b) => a.id.localeCompare(b.id));
    },
  };
}
```

storage-repository.mjs · JavaScript — চলবে: Node.js

```
// একই চুক্তি, কিন্তু Web Storage-এ (localStorage বা Test-এর নকল Storage)।
// Adapter: Storage-এর getItem/setItem-কে Repository-র ভাষায় রূপান্তর।
export function createStorageRepository(storage, prefix = 'visaflow:') {
  const keys = () =>
    Array.from({ length: storage.length }, (_, i) => storage.key(i))
      .filter((key) => key.startsWith(prefix));
```

```
  function read(key) {
    try {
      return JSON.parse(storage.getItem(key));
    } catch {
      return null;
    }
  }

  return {
    save(application) {
      storage.setItem(prefix + application.id, JSON.stringify(application));
    },
    get(id) {
      return read(prefix + id);
    },
    list() {
      return keys()
        .map(read)
        .filter((item) => item !== null)
        .toSorted((a, b) => a.id.localeCompare(b.id));
    },
  };
}
```

দুটিই একই চুক্তি মানে, এবং দুটিই Copy দেয়-নেয় (`structuredClone`, JSON)—ফেরত পাওয়া Object বদলালে সংরক্ষিত তথ্য বদলায় না। Storage Adapter `localStorage` সরাসরি ব্যবহার না করে একটি `storage` Parameter নেয়; Browser-এ `localStorage`, Test-এ একটি ছোট নকল। নিজস্ব Prefix-এর বাইরের Key এবং ভাঙা JSON উপেক্ষিত, কারণ `localStorage` অন্য Code বা ব্যবহারকারী বদলাতে পারেন।

UI ও Browser-এর Composition Root

view.mjs · JavaScript — চলবে: Browser

```
import { availableActions } from '../domain/workflow.mjs';

const LABELS = {
  submit: 'জমা দিন',
  requestDocuments: 'কাগজ চাই',
  resubmit: 'আবার জমা',
  approve: 'অনুমোদন',
  reject: 'প্রত্যাখ্যান',
  withdraw: 'প্রত্যাহার',
};

// View শুধু আঁকে; কোন কাজ চলবে তা State Machine বলে, কী ঘটবে তা onAction।
export function renderApplications(list, applications, onAction) {
  list.replaceChildren(...applications.map((application) => {
```

```
    const row = document.createElement('li');
    row.dataset.id = application.id;
    const text = document.createElement('span');
    text.textContent = `${application.id} · ${application.applicant} · `
      + `${application.country} ${application.visaType} · ৳${application.fee}`
      + ` · ${application.status}`;
    row.append(text);
    for (const action of availableActions(application.status)) {
      const button = document.createElement('button');
      button.type = 'button';
      button.className = `act-${action}`;
      button.textContent = LABELS[action] ?? action;
      button.addEventListener('click', () => onAction(application.id, action));
      row.append(button);
    }
    return row;
  }));
}

export function renderSummary(element, { counts, approvedFees }) {
  const parts = Object.entries(counts).map(([status, n]) => `${status} ${n}`);
  element.textContent = `${parts.join(', ') || 'কোনো আবেদন নেই'}`
    + ` · অনুমোদিত Fee ৳${approvedFees}`;
}
```

main.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser

```
// Composition Root (Browser): একমাত্র জায়গা যেখানে সব অংশ জোড়া লাগে।
import { createVisaService } from './app/service.mjs';
import { createEventBus } from './app/events.mjs';
import { createStorageRepository } from './infra/storage-repository.mjs';
import { renderApplications, renderSummary } from './ui/view.mjs';
import { sequenceAfter } from './app/ids.mjs';

const $ = (selector) => document.querySelector(selector);
const repository = createStorageRepository(localStorage);
const events = createEventBus({ onError: (error) => console.error(error) });
const service = createVisaService({
  repository,
  events,
  nextId: sequenceAfter('VF', repository.list().map((item) => item.id)),
});

function refresh() {
  renderApplications($('#applications'), service.list(), handleAction);
  renderSummary($('#summary'), service.summary());
}

function handleAction(id, action) {
```

```
  try {
    service.act(id, action);
  } catch (error) {
    $('#message').textContent = error.message;
  }
}

// ...
```

main.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser

```
// ...

// তিনজন Subscriber, একে অপরকে না চিনে: Console Log, পর্দা, বার্তা।
events.on('application:created', (e) => {
  console.log(`নতুন ${e.id} ৳${e.fee}`);
});
events.on('application:changed', (e) => {
  console.log(`${e.id}: ${e.from} থেকে ${e.to}`);
});
for (const type of ['application:created', 'application:changed']) {
  events.on(type, refresh);
  events.on(type, () => {
    $('#message').textContent = '';
  });
}

$('#new-application').addEventListener('submit', (event) => {
  event.preventDefault();
  try {
    service.create({
      applicant: $('#applicant').value,
      country: $('#country').value,
      visaType: $('#visa-type').value,
      travelers: Number($('#travelers').value),
      express: $('#express').checked,
    });
    event.target.reset();
  } catch (error) {
    $('#message').textContent = error.message;
  }
});

refresh();
```

`view.mjs` কোন Button দেখাবে তা নিজে ঠিক করে না; `availableActions` State Machine থেকে জানায়। সব লেখা `textContent` দিয়ে। `main.mjs` একমাত্র File যেখানে `localStorage`, DOM ও Service একসঙ্গে দেখা যায়। Event-এর তিনজন Subscriber (Console Log, পর্দা নতুন করে আঁকা, পুরোনো বার্তা মোছা) একে অপরকে চেনে না;

একটি নতুন Subscriber (যেমন একটি Notification) যোগ করতে শুধু একটি `events.on` ।

index.html · HTML চলবে: Browser

```
<!doctype html>
<html lang="bn">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>VisaFlow</title>
  <link rel="stylesheet" href="style.css">
  <script type="module" src="src/main.mjs"></script>
</head>
<body>
  <h1>VisaFlow <small>শিক্ষামূলক Demo</small></h1>
  <form id="new-application">
    <label>আবেদনকারী <input id="applicant" required></label>
    <label>দেশ <select id="country">
      <option>Thailand</option><option>Malaysia</option>
      <option>Japan</option><option>Turkey</option>
    </select></label>
    <label>ধরন <select id="visa-type">
      <option value="tourist">Tourist</option>
      <option value="business">Business</option>
      <option value="student">Student</option>
    </select></label>
    <label>যাত্রী <input id="travelers" type="number" value="1"></label>
    <label><input id="express" type="checkbox"> Express</label>
    <button id="create">আবেদন তৈরি</button>
  </form>
  <p id="message" role="alert"></p>
  <p id="summary"></p>
  <ul id="applications"></ul>
</body>
</html>
```

style.css · CSS চলবে: Browser

```
body {
  font-family: system-ui, "Nirmala UI", sans-serif;
  max-width: 44rem;
  margin: 2rem auto;
  padding: 0 1rem;
}

form {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem 1rem;
  align-items: end;
```

```
}

#applications li {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  align-items: center;
  margin-bottom: 0.5rem;
}

#message {
  color: #b91c1c;
}
```

Node.js-এর Composition Root: demo.mjs

demo.mjs · JavaScript — চলবে: Node.js

```
// Composition Root (Node.js): একই Service, কিন্তু Memory Repository
// ও নির্দিষ্ট ঘড়ি।
import { createVisaService } from './src/app/service.mjs';
import { createEventBus } from './src/app/events.mjs';
import { createSequence } from './src/app/ids.mjs';
import { createMemoryRepository } from './src/infra/memory-repository.mjs';

const events = createEventBus();
const service = createVisaService({
  repository: createMemoryRepository(),
  events,
  nextId: createSequence('VF'),
  clock: () => new Date('2026-10-15T10:00:00Z'),
});
events.on('application:changed', (e) => {
  console.log(`  ${e.id}: ${e.from} থেকে ${e.to}`);
});

const rafi = service.create({ applicant: '  Rafi   Ahmed ', country: 'Thailand',
  visaType: 'tourist', travelers: 2 });
const mina = service.create({ applicant: 'Mina Akter', country: 'Japan',
  visaType: 'business', travelers: 1, express: true });
const sara = service.create({ applicant: 'Sara', country: 'Turkey',
  visaType: 'student', travelers: 1 });
console.log(rafi.applicant, rafi.fee, mina.fee, sara.fee);

service.act(rafi.id, 'submit');
service.act(rafi.id, 'approve');
service.act(mina.id, 'submit');
service.act(mina.id, 'requestDocuments');
service.act(sara.id, 'withdraw');
```

```
for (const [id, action] of [[sara.id, 'submit'], ['VF-999', 'submit']]) {
  try {
    service.act(id, action);
  } catch (error) {
    console.log(`  ${error.code}: ${error.message}`);
  }
}
const { counts, approvedFees } = service.summary();
console.log(counts, approvedFees);
```

Output

```
Rafi Ahmed 6000 9100 2500
  VF-001: draft থেকে submitted
  VF-001: submitted থেকে approved
  VF-002: draft থেকে submitted
  VF-002: submitted থেকে documents-requested
  VF-003: draft থেকে withdrawn
  INVALID_TRANSITION: withdrawn অবস্থায় "submit" করা যায় না
  NOT_FOUND: VF-999 নামে কোনো আবেদন নেই
{ approved: 1, 'documents-requested': 1, withdrawn: 1 } 6000
```

একই Service, একই Domain, কিন্তু Repository Memory-তে এবং ঘড়ি স্থির। Rafi-র নামের বাড়তি ফাঁকা Factory সরিয়েছে; Mina-র Business Express Fee (৫৫০০ + ১০০০) × ১.৪ = ৯১০০; প্রত্যাহার করা আবেদন আর জমা দেওয়া যায় না, এবং অজানা ID-র জন্য স্পষ্ট `NOT_FOUND`।

Test-এর কৌশল

Terminal

```
cd visa-flow
node --test
node demo.mjs
python -m http.server 8000
```

শেষ লাইনটি Browser UI-র জন্য: ES Module `file://` থেকে চলে না, তাই একটি Static Server লাগে (অধ্যায় ১২); Python না থাকলে যেকোনো Static Server, এবং Browser-এ `http://localhost:8000`। `node --test` ৩১টি Test চালায়, চারটি ভাগে:

File	কী যাচাই করে	Test
`domain.test.mjs`	Factory, প্রতিটি Fee Strategy ও Decorator, সম্পূর্ণ ও অবৈধ Workflow, Immutability	৮
`service.test.mjs`	Use Case, Event-এর ক্রম, ব্যর্থতায় কোনো চিহ্ন বা নষ্ট ID নেই, ID-র ক্রম, DI-তে নকল Repository, Event Bus-এর Error বিচ্ছিন্নতা	৯

File	কী যাচাই করে	Test
`repositories.test.mjs`	একই চার Contract Test দুটি Adapter-এ, এবং Storage-এর ভাঙা তথ্য	৯
`architecture.test.mjs`	প্রতিটি স্তরের Import-এর নিয়ম, Composition Root	৫

Contract Test-এর ধারণাটি গুরুত্বপূর্ণ: একই Test-এর তালিকা প্রতিটি Repository-তে চলে, তাই ভবিষ্যতে একটি API Repository যোগ করলে শুধু একটি লাইন যোগ করে প্রমাণ করা যায় যে সেটিও একই আচরণ করে। এছাড়া এই বইয়ের QA-তে আসল Chrome-এ UI-র একটি End-to-end যাচাই চলেছে: Form দিয়ে আবেদন তৈরি, Button দিয়ে Workflow এগোনো, অবৈধ ইনপুটের বার্তা, এবং Page আবার লোড করার পর `localStorage` থেকে তথ্য ফিরে আসা।

এই End-to-end যাচাই প্রথমবার চালানোর সময় দুটি আসল ত্রুটি ধরেছিল, যা ২৯টি Unit Test-এর কোনোটি ধরেনি। প্রথমত, Service যাচাইয়ের **আগে** `nextId()` ডাকত, তাই প্রতিটি ব্যর্থ চেষ্টা একটি ID নষ্ট করত (VF-001-এর পরে সরাসরি VF-004)। দ্বিতীয়ত, Page আবার লোড হলে ID-র ক্রম শুরু হতো "সংরক্ষিত আবেদনের সংখ্যা + ১" থেকে; ID-তে ফাঁক থাকলে এটি একটি বিদ্যমান ID আবার দিত, এবং নতুন আবেদন পুরোনোটিকে মুছে ফেলত। সমাধান দুটি এখন Code-এ (Factory-র `newId`, এবং `ids.mjs`-এর `sequenceAfter`), এবং প্রতিটির জন্য একটি নতুন Unit Test যোগ হয়েছে, যাতে ত্রুটি ফিরে না আসে। শিক্ষা: প্রতিটি স্তর আলাদাভাবে সঠিক হলেও, জোড়া লাগানোর জায়গায় (Composition Root ও স্তরের সীমানা) ত্রুটি লুকাতে পারে—তাই Testing Pyramid-এর ওপরের স্তরেও কিছু Test দরকার (অধ্যায় ১৬)।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Browser-এ কিছুই আঁকা হয় না	Page `file://` থেকে খোলা	Static Server
Architecture Test ব্যর্থ	কোনো স্তর নিষিদ্ধ Import করছে	নির্ভরতা Inject করুন, Import নয়
Button দেখা যায় কিন্তু কাজ হয় না	Workflow-এ কাজটির নাম ভুল	`availableActions` ও `WORKFLOW` মেলান
পুরোনো তথ্য অদ্ভুত	`localStorage`-এ পুরোনো Format	Prefix বদলান বা Storage মুছুন
Event-এর পর পর্দা হালনাগাদ হয় না	Subscriber নিবন্ধিত নয়	`events.count(type)` দেখুন

২১.১১ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
একটি Function-এ DOM, নিয়ম ও Storage	কোনোটি আলাদা বদলানো বা Test করা যায় না	স্তরে ভাগ
প্রয়োজন ছাড়া Pattern	অপ্রয়োজনীয় জটিলতা	দ্বিতীয়-তৃতীয় প্রয়োজনে

ভুল	কেন সমস্যা	সমাধান
Domain-এ `localStorage`, `fetch` বা `document`	Domain পরিবেশ-নির্ভর	Repository ও Inject
Service-এ ব্যবসার নিয়ম	নিয়ম ছড়িয়ে পড়ে	Domain-এ
ভেতরে `new Date()` বা `Math.random()`	Test অনিশ্চিত	ঘড়ি ও ID Inject
Status-এর `if/else` ছড়ানো	অসামঞ্জস্য	টেবিলভিত্তিক State Machine
Event দিয়ে আদেশ দেওয়া	প্রবাহ লুকায়	সরাসরি Function Call
Unsubscribe না করা	Memory Leak	`on`-এর ফেরত Function
অবস্থাযুক্ত Module-Singleton	Test একে অপরকে প্রভাবিত করে	Factory ও Inject
স্তরের নিয়ম শুধু Document-এ	সময়ের সঙ্গে ভাঙে	Architecture Test

২১.১২ Debugging নির্দেশিকা

১. **কোন স্তরে?** ভুল হিসাব হলে Domain-এর Unit Test; ভুল ক্রম হলে Service; ভুল প্রদর্শন হলে View। স্তর আলাদা থাকায় খোঁজার জায়গা ছোট।

২. **Event-এর প্রবাহ:** অস্থায়ীভাবে একটি Subscriber যোগ করুন যা প্রতিটি Event Log করে; কে আগে, কে পরে চলে দেখুন।

৩. **State Machine:** `INVALID_TRANSITION` বার্তায় বর্তমান অবস্থা ও কাজের নাম থাকে; টেবিলের সঙ্গে মেলান।

৪. **Repository:** একই Contract Test দুটি Adapter-এ চালান; একটিতে ব্যর্থ হলে সমস্যা সেই Adapter-এ।

৫. **Composition Root:** "undefined is not a function" প্রায়ই একটি নির্ভরতা Inject না করার ফল; Service-এর Parameter মিলিয়ে দেখুন।

২১.১৩ পেশাদার অভ্যাস

- **যা একসঙ্গে বদলায়, তা একসঙ্গে রাখুন; যা ভিন্ন কারণে বদলায়, তা আলাদা।**
- **নির্ভরতা ভেতরের দিকে;** Domain সবচেয়ে স্থির ও পরিবেশ-মুক্ত।
- **Interface ছোট রাখুন:** Repository-র তিনটি Method যথেষ্ট হলে দশটি নয়।
- **Composition-এ পছন্দ, Inheritance-এ সংযম।**
- **একটি Composition Root;** বাকি Code শুধু Parameter পায়।
- **Pattern-এর নাম Code-এ ব্যবহার করুন** (`createStorageRepository`, `withExpress`), যাতে পাঠক সঙ্গে সঙ্গে চেনেন।
- **Architecture-এর সিদ্ধান্ত লিখে রাখুন** (ছোট Architecture Decision Record): কী, কেন, কোন বিকল্প বাদ।

- **নিয়ম Test দিয়ে পাহারা দিন,** শুধু Document দিয়ে নয়।

২১.১৪ অনুশীলনী

প্রাথমিক স্তর

১. প্রতিটি Code কোন Pattern: (ক) `button.addEventListener('click', fn)`, (খ) `createApplication(input, context)`, (গ) `sort((a, b) => a.price - b.price)`-এ তুলনার Function দেওয়া, (ঘ) `withExpress(policy)`।

২. VisaFlow-এর একই আকারে একটি `transit` Fee Strategy লিখুন: যাত্রীপ্রতি ১,২০০ টাকা, সর্বোচ্চ ৪ জন; Express Decorator দিয়ে ২ জনের Fee দেখান।

৩. একটি Support Ticket-এর জন্য অবস্থা ও কাজের টেবিল তৈরি করুন: open, in-progress, resolved, closed।

৪. Event Bus-এ `once(type, handler)` যোগ করুন, যা প্রথমবারের পরেই নিজে সরে যায়।

৫. VisaFlow-এ কেন শুধু `main.mjs` ও `demo.mjs` Infrastructure Import করে? দুই বাক্যে লিখুন।

মধ্যম স্তর

৬. একটি Logging Decorator Repository লিখুন: যেকোনো Repository মুড়ে প্রতিটি `save` ও `get` গণনা করে, এবং আচরণ অপরিবর্তিত রাখে।

৭. VisaFlow-এর Workflow থেকে একটি Mermaid-ধাঁচের লেখা Diagram তৈরি করুন (`draft --submit--> submitted` ধরনের লাইন), যাতে Diagram সব সময় Code-এর সঙ্গে মেলে।

৮. Contract Test-এর মতো একটি তৃতীয় Repository (একটি JSON String-এ সব আবেদন রাখে) লিখুন, এবং দেখান যে Service তার সঙ্গেও কাজ করে।

৯. একটি Audit Subscriber লিখুন, যা প্রতিটি `application:changed` Event একটি তালিকায় রাখে এবং ৩টির পরে নিজেকে Unsubscribe করে।

চ্যালেঞ্জ স্তর

১০. আবেদনের ইতিহাস থেকে "আগের অবস্থায় ফেরা" (Undo) প্রয়োগ করুন: শেষ Entry বাদ দিয়ে নতুন Object, কিন্তু চূড়ান্ত অবস্থা থেকে Undo নিষিদ্ধ।

১১. একটি Strategy Registry লিখুন, যা Runtime-এ নতুন Fee Strategy নিবন্ধন করে, কিন্তু আকার (`label`, `maxTravelers`, `fee`) যাচাই না হলে প্রত্যাখ্যান করে।

১২. নিচের জট পাকানো Function-কে Domain (বিশুদ্ধ হিসাব) ও বাকি অংশে ভাগ করুন, এবং বিশুদ্ধ অংশটি Browser ছাড়াই Test করুন: `function onSubmit() { const n = Number(document.querySelector('#n').value); const fee = n > 3 ? n * 2800 : n *`

```
3000; localStorage.setItem('fee', fee);
document.querySelector('#out').textContent = fee; }
```

২১.১৫ অনুশীলনীর সমাধান

সমাধান ১ — (ক) Observer: Browser Publisher, আপনার Function Subscriber। (খ) Factory: যাচাইসহ তৈরি। (গ) Strategy: তুলনার নিয়ম Parameter হিসেবে বিনিময়যোগ্য। (ঘ) Decorator: একটি Strategy মুড়ে আচরণ বাড়ানো।

সমাধান ২

transit.mjs · JavaScript — চলবে: Node.js

```
import { withExpress } from '../../project/visa-flow/src/domain/fees.mjs';

const transit = {
  label: 'Transit',
  maxTravelers: 4,
  fee: ({ travelers }) => 1200 * travelers,
};

const two = { travelers: 2 };
console.log(transit.fee(two), withExpress(transit).fee(two));
console.log(withExpress(transit).maxTravelers);
```

Output

```
2400 3360
4
```

Decorator নতুন Strategy-র সঙ্গেও কোনো পরিবর্তন ছাড়াই কাজ করেছে, কারণ তা শুধু চুক্তি (`fee`) জানে। Project-এ স্থায়ীভাবে যোগ করতে `FEE_POLICIES` -এ একটি লাইন।

সমাধান ৩

অবস্থা	কাজ	নতুন অবস্থা
open	start / close	in-progress / closed
in-progress	resolve / pause	resolved / open
resolved	reopen / close	in-progress / closed
closed	কোনোটি নয়	চূড়ান্ত

সমাধান ৪

once.mjs · JavaScript চলবে: Node.js

```
import { createEventBus } from '../../project/visa-flow/src/app/events.mjs';

function once(bus, type, handler) {
  const off = bus.on(type, (payload) => {
    off();
    handler(payload);
  });
  return off;
}

const bus = createEventBus();
once(bus, 'ready', (n) => console.log(`প্রথমবার: ${n}`));
bus.emit('ready', 1);
bus.emit('ready', 2);
console.log(bus.count('ready'));
```

Output

```
প্রথমবার: 1
0
```

Event Bus না বদলে তার ওপর একটি ছোট Function—Composition।

সমাধান ৫ — Infrastructure পরিবেশ-নির্ভর (`localStorage` শুধু Browser-এ), তাই যে Code তা Import করে, সেটি সেই পরিবেশে আটকে যায়। Import শুধু Composition Root-এ থাকায় Domain ও Service যেকোনো পরিবেশে চলে এবং নকল দিয়ে Test করা যায়।

সমাধান ৬

logging-repo.mjs · JavaScript চলবে: Node.js

```
import { createMemoryRepository }
  from '../../project/visa-flow/src/infra/memory-repository.mjs';

function withCounting(repository) {
  const counts = { save: 0, get: 0 };
  return {
    counts,
    save(item) {
      counts.save += 1;
      return repository.save(item);
    },
```

```
    get(id) {
      counts.get += 1;
      return repository.get(id);
    },
    list: () => repository.list(),
  };
}

const repository = withCounting(createMemoryRepository());
repository.save({ id: 'VF-001', status: 'draft' });
repository.get('VF-001');
repository.get('VF-002');
console.log(repository.counts, repository.list().length);
```

Output

```
{ save: 1, get: 2 } 1
```

সমাধান ৭

diagram.mjs · JavaScript — চলবে: Node.js

```
import { WORKFLOW } from '../../project/visa-flow/src/domain/workflow.mjs';

const lines = Object.entries(WORKFLOW).flatMap(([from, moves]) =>
  Object.entries(moves).map(([action, to]) => `  ${from} --${action}--> ${to}`));

console.log('stateDiagram');
console.log(lines.join('\n'));
```

Output

```
stateDiagram
  draft --submit--> submitted
  draft --withdraw--> withdrawn
  submitted --requestDocuments--> documents-requested
  submitted --approve--> approved
  submitted --reject--> rejected
  submitted --withdraw--> withdrawn
  documents-requested --resubmit--> submitted
  documents-requested --withdraw--> withdrawn
```

Diagram Code থেকে তৈরি, তাই Workflow বদলালে Diagram নিজেই বদলায়—হাতে আঁকা Diagram যেমন পুরোনো হয়ে যায়, এটি তেমন হয় না।

সমাধান ৮

json-repo.mjs · JavaScript চলবে: Node.js

```
import { createVisaService } from '../../project/visa-flow/src/app/service.mjs';
import { createEventBus } from '../../project/visa-flow/src/app/events.mjs';
import { createSequence } from '../../project/visa-flow/src/app/ids.mjs';

function createJsonRepository() {
  let json = '{}';
  const all = () => JSON.parse(json);
  return {
    save(item) {
      json = JSON.stringify({ ...all(), [item.id]: item });
    },
    get: (id) => (Object.hasOwn(all(), id) ? all()[id] : null),
    list: () => Object.values(all()).toSorted((a, b) => a.id.localeCompare(b.id)),
    size: () => json.length,
  };
}

const repository = createJsonRepository();
const service = createVisaService({
  repository, events: createEventBus(), nextId: createSequence('VF'),
});
service.create({
  applicant: 'Sara', country: 'Japan', visaType: 'student', travelers: 1,
});
service.act('VF-001', 'submit');
const rows = service.list().map((a) => `${a.id} ${a.status}`);
console.log(rows, repository.size() > 0);
```

Output

```
[ 'VF-001 submitted' ] true
```

Service-এর একটি লাইনও বদলাতে হয়নি। পুরো প্রমাণের জন্য এটিকে `repositories.test.mjs`-এর `adapters` তালিকায় যোগ করলে চারটি Contract Test এর ওপরও চলবে।

সমাধান ৯

audit.mjs · JavaScript চলবে: Node.js

```
import { createEventBus } from '../../project/visa-flow/src/app/events.mjs';

const bus = createEventBus();
const audit = [];
const stop = bus.on('application:changed', (event) => {
  audit.push(`${event.id}:${event.to}`);
  if (audit.length === 3) {
    stop();
  }
});

for (const to of ['submitted', 'documents-requested', 'submitted', 'approved']) {
  bus.emit('application:changed', { id: 'VF-001', to });
}
console.log(audit.join(' | '), bus.count('application:changed'));
```

Output

```
VF-001:submitted | VF-001:documents-requested | VF-001:submitted 0
```

সমাধান ১০

undo.mjs · JavaScript চলবে: Node.js

```
import { createApplication }
  from '../../project/visa-flow/src/domain/application.mjs';
import { applyAction, isFinal }
  from '../../project/visa-flow/src/domain/workflow.mjs';

function undo(application) {
  if (isFinal(application.status) || application.history.length < 2) {
    throw new Error(`${application.status} থেকে Undo করা যায় না`);
  }
  const history = application.history.slice(0, -1);
  const { status } = history.at(-1);
  return Object.freeze({ ...application, status, history });
}

const now = new Date('2026-10-15T10:00:00Z');
const draft = createApplication(
  { applicant: 'Rafi', country: 'Malaysia', visaType: 'tourist', travelers: 1 },
  { newId: () => 'VF-001', now });
const submitted = applyAction(draft, 'submit', now);
const asked = applyAction(submitted, 'requestDocuments', now);
```

```
console.log(undo(asked).status, undo(undo(asked)).status);
for (const app of [draft, applyAction(draft, 'withdraw', now)]) {
  try {
    undo(app);
  } catch (error) {
    console.log(error.message);
  }
}
```

Output

```
submitted draft
draft থেকে Undo করা যায় না
withdrawn থেকে Undo করা যায় না
```

ইতিহাস Immutable হওয়ায় Undo সহজ: শেষ Entry বাদ দিলেই আগের অবস্থা। চূড়ান্ত অবস্থা থেকে Undo নিষিদ্ধ করা একটি ব্যবসার সিদ্ধান্ত, তাই এটি Domain-এ থাকা উচিত।

সমাধান ১১

registry.mjs · JavaScript — চলবে: Node.js

```
function createRegistry() {
  const strategies = new Map();
  return {
    register(name, strategy) {
      const valid = typeof strategy?.label === 'string'
        && Number.isInteger(strategy.maxTravelers) && strategy.maxTravelers > 0
        && typeof strategy.fee === 'function';
      if (!valid) {
        throw new TypeError(`${name}: label, maxTravelers ও fee প্রয়োজন`);
      }
      if (strategies.has(name)) {
        throw new Error(`${name} আগেই নিবন্ধিত`);
      }
      strategies.set(name, Object.freeze({ ...strategy }));
    },
    get: (name) => strategies.get(name) ?? null,
    names: () => [...strategies.keys()],
  };
}

const registry = createRegistry();
registry.register('transit',
  { label: 'Transit', maxTravelers: 4, fee: () => 1200 });
for (const [name, strategy] of [['bad', { label: 'Bad' }], ['transit', {}]]) {
  try {
```

```
    registry.register(name, strategy);
  } catch (error) {
    console.log(error.name, error.message);
  }
}
console.log(registry.names(), registry.get('transit').fee());
```

Output

```
TypeError bad: label, maxTravelers ও fee প্রয়োজন
TypeError transit: label, maxTravelers ও fee প্রয়োজন
[ 'transit' ] 1200
```

`Map` উত্তরাধিকারের Key-এর সমস্যা এড়ায়, এবং আকার যাচাই নিশ্চিত করে যে ভুল Strategy পরে, ব্যবহারের মুহূর্তে নয়, নিবন্ধনের সময়ই ধরা পড়ে।

সমাধান ১২

refactor.mjs · JavaScript — চলবে: Node.js

```
// Domain: বিশুদ্ধ, Browser ছাড়াই Test করা যায়।
export function groupFee(travelers) {
  if (!Number.isInteger(travelers) || travelers < 1) {
    throw new RangeError('যাত্রী একটি ধনাত্মক পূর্ণসংখ্যা');
  }
  return travelers * (travelers > 3 ? 2800 : 3000);
}

// UI ও Storage: পাতলা আবরণ, নির্ভরতা Parameter-এ।
export function onSubmit({ read, save, show }) {
  const fee = groupFee(Number(read('#n')));
  save('fee', fee);
  show('#out', fee);
}

const shown = [];
const show = (_, value) => shown.push(value);
onSubmit({ read: () => '4', save: () => {}, show });
console.log([1, 3, 4].map(groupFee), shown);
```

Output

```
[ 3000, 9000, 11200 ] [ 11200 ]
```

Browser-এ `onSubmit`-কে `read: (s) => document.querySelector(s).value`, `save: (k, v) => localStorage.setItem(k, v)` ইত্যাদি দিয়ে ডাকা হবে। নিয়মটি এখন এক জায়গায়, নামযুক্ত, এবং Test-যোগ্য; অবৈধ ইনপুট (`NaN`) আর নিঃশব্দে `NaN` Fee হিসেবে সংরক্ষিত হয় না।

২১.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. একই কাজের একাধিক বিনিময়যোগ্য নিয়ম, চলার সময় বেছে নেওয়া—কোন Pattern?

ক) Observer

খ) Strategy

গ) Singleton

ঘ) Repository

প্রশ্ন ২. Factory Function-এর প্রধান সুবিধা?

ক) তৈরির নিয়ম ও যাচাই এক জায়গায়, ব্যবহারকারী Implementation জানে না

খ) Code দ্রুত চলে

গ) Memory কম লাগে

ঘ) Class ব্যবহার বাধ্যতামূলক হয়

প্রশ্ন ৩. Observer Pattern-এর একটি ঝুঁকি?

ক) Subscriber যোগ করা যায় না

খ) Publisher সব Subscriber-কে চেনে

গ) প্রবাহ লুকিয়ে যায় এবং Unsubscribe না করলে Leak

ঘ) একটির বেশি Subscriber থাকতে পারে না

প্রশ্ন ৪. Dependency Injection-এর মূল অর্থ?

ক) সব Module Global-এ রাখা

খ) Library Install করা

গ) সব Code এক File-এ

ঘ) নির্ভরতা নিজে তৈরি না করে বাইরে থেকে পাওয়া

প্রশ্ন ৫. Composition Root কী?

ক) Project-এর Root Folder

খ) একমাত্র জায়গা যেখানে সব নির্ভরতা তৈরি ও জোড়া লাগে

গ) সবচেয়ে বড় Class

ঘ) DOM-এর Root Element

প্রশ্ন ৬. Layered Architecture-এ Domain স্তর কীসের ওপর নির্ভর করা উচিত?

ক) কোনো বাইরের স্তরের ওপর নয়

খ) UI-এর ওপর

গ) `localStorage`-এর ওপর

ঘ) Infrastructure-এর ওপর

প্রশ্ন ৭. VisaFlow-এ `withdrawn` অবস্থার আবেদনে `submit` করলে কী হয়?

ক) আবেদন জমা হয়

খ) কিছুই ঘটে না, নিঃশব্দে

গ) `INVALID_TRANSITION` DomainError, কিছু সংরক্ষিত হয় না

ঘ) অবস্থা `draft` হয়

প্রশ্ন ৮. Contract Test-এর উদ্দেশ্য?

ক) একই চুক্তির প্রতিটি Implementation একই আচরণ করে কি না যাচাই

খ) আইনি চুক্তি যাচাই

গ) শুধু একটি Repository Test

ঘ) UI-এর রং যাচাই

প্রশ্ন ৯. ES Module-এর স্তরে রাখা অবস্থা (Module-Singleton)-এর ঝুঁকি?

ক) প্রতিটি Import নতুন Copy পায়

খ) Module একাধিকবার চলে

গ) Browser-এ কাজ করে না

ঘ) অবস্থা Test-এর মধ্যে ভাগ হয়ে যায় এবং বদলানো কঠিন

প্রশ্ন ১০. কখন একটি Design Pattern প্রয়োগ করা উচিত?

ক) সব সময়, যত বেশি তত ভালো

খ) একই ধরনের প্রয়োজন বারবার এলে, যখন তা জটিলতা কমায়

গ) শুধু বড় দলে

ঘ) কখনো নয়

২১.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–খ · ৬–ক · ৭–গ · ৮–ক · ৯–ঘ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Fee-র নিয়ম বা ছাড়ের নিয়ম।
২	ক	`createApplication` অবৈধ আবেদন তৈরি হতে দেয় না।
৩	গ	তাই `on` Unsubscribe ফেরত দেয় এবং Handler আলাদা `try/catch`-এ।
৪	ঘ	কোনো Framework লাগে না; একটি Parameter-ই যথেষ্ট।
৫	খ	`main.mjs` ও `demo.mjs`।
৬	ক	তাই Domain সব পরিবেশে চলে।
৭	গ	`demo.mjs`-এর Output-এ দেখা যায়।
৮	ক	দুটি Repository-তে একই চারটি Test।
৯	ঘ	এক Test-এর পরিবর্তন অন্যটিতে রয়ে যায়।
১০	খ	Pattern সমাধান, লক্ষ্য নয়।

২১.১৮ অধ্যায়ের সারসংক্ষেপ

- Architecture-এর লক্ষ্য পরিবর্তন সস্তা রাখা: ভিন্ন কারণে বদলানো অংশ আলাদা।
- Pattern নামযুক্ত সমাধান; JavaScript-এ Function ও Object দিয়ে হালকা রূপে; প্রয়োজন এলে তবেই।
- Factory তৈরির নিয়ম লুকায় ও অবৈধ Object আটকায়; Strategy if/else-এর শিকল ভাঙে; Decorator আচরণ বাড়ায় মূল না বদলে।
- Observer ঘোষণাকারী ও শ্রোতাকে আলাদা রাখে; Unsubscribe ও Error-এর বিচ্ছিন্নতা জরুরি।
- Repository সংরক্ষণ লুকায়, Adapter বাইরের Interface রূপান্তর করে; Contract Test সব Implementation-কে এক মানে রাখে।
- Dependency Injection ও একটি Composition Root: বাকি Code শুধু Parameter পায়; ঘড়ি ও ID-ও নির্ভরতা।
- State Machine Workflow-কে তথ্যে রূপান্তর করে; UI সেখান থেকেই কাজের তালিকা পায়।
- নির্ভরতা ভেতরের দিকে; Domain পরিবেশ-মুক্ত; নিয়ম Architecture Test-এ পাহারা।

পরের অধ্যায়ে Testing আরও গভীরে: Unit, Integration ও End-to-end Test-এর কৌশল, Mocking, এবং Test-Driven Development।

২১.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Design Pattern	একটি পরিচিত সমস্যার নামযুক্ত, পুনর্ব্যবহারযোগ্য সমাধান।

পরিভাষা	অর্থ
Factory	Object তৈরির নিয়ম লুকানো Function।
Strategy	বিনিময়যোগ্য নিয়ম, একই আকারে।
Decorator	একটি Object মুড়ে আচরণ বাড়ানো।
Observer	ঘটনার ঘোষণা ও স্বাধীন শ্রোতা।
Repository	সংরক্ষণের বিস্তারিত লুকানো চুক্তি।
Adapter	একটি Interface-কে আরেকটিতে রূপান্তর।
Dependency Injection	নির্ভরতা বাইরে থেকে দেওয়া।
Composition Root	যেখানে সব নির্ভরতা জোড়া লাগে।
State Machine	অবস্থা ও বৈধ পরিবর্তনের টেবিল।
Layered Architecture	দায়িত্বভিত্তিক স্তর ও নির্ভরতার দিক।
Contract Test	একই চুক্তির সব Implementation-এ একই Test।

২১.২০ আরও পড়ুন

- **Refactoring.Guru — Design Patterns:** `https://refactoring.guru/design-patterns` — প্রতিটি Pattern-এর ব্যাখ্যা ও উদাহরণ।
- **Patterns.dev:** `https://www.patterns.dev/` — JavaScript-কেন্দ্রিক Design ও Rendering Pattern।
- **Martin Fowler — Inversion of Control Containers and the Dependency Injection pattern:** `https://martinfowler.com/articles/injection.html` — DI-এর মূল প্রবন্ধ।
- **Alistair Cockburn — Hexagonal Architecture:** `https://alistair.cockburn.us/hexagonal-architecture/` — Ports and Adapters।
- **MDN — JavaScript modules:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules` — Module-এর আচরণ।